Triton-based INT4 Quantized KV Cache for Efficient LLM Inference

John Chmura Akash Madhu

May 6, 2026

1 Project Type

This project is mainly focused on **Engineering/measurement** and **Systemization/reproduction**. We engineer a custom Triton kernel for INT4 KV cache quantization and measure its impact on large language models (LLMs) like LLaMA 3.1 (8B and 70B) and GPT-2 across multi-GPU setups.

2 Motivation and Problem Statement

In modern Large Language Models (LLMs), the Key-Value (KV) cache grows linearly with the sequence length and batch size. During the autoregressive decoding phase, memory bandwidth becomes the primary bottleneck rather than compute, as the entire KV cache must be loaded from High Bandwidth Memory (HBM) to SRAM for every generated token. While quantizing the KV cache to INT4 can theoretically reduce this memory footprint by up to 4×, naively integrating this into standard frameworks (e.g., Hugging Face using PyTorch SDPA) requires first dequantizing the INT4 cache back into a massive FP16/BF16 tensor in HBM before computing attention. This two-step process severely limits the memory bandwidth savings during decode. This project addresses this problem by building a custom Triton kernel that fuses on-the-fly INT4 dequantization directly with the attention computation inside SRAM. This enables true memory bandwidth acceleration, which is crucial for efficient deep learning as it allows for larger batch sizes, longer context lengths, and faster decode speeds on AI accelerators.

3 Background and Related Work

3.1 The Memory Wall in LLM Inference

While methods like PagedAttention [1] (used in vLLM) optimize the memory fragmentation of the KV cache, and quantization techniques like AWQ [2] and GPTQ [3] compress the size of the weights, the fundamental calculation of attention remains bounded by memory bandwidth. Specifically, reading large uncompressed KV caches into the GPU's compute units during the decode phase creates a severe bottleneck.

3.2 FlashAttention and Tiling

Standard attention computation materializes a large $N \times N$ attention matrix in the GPU's High Bandwidth Memory (HBM). FlashAttention [5] solves this bandwidth bottleneck by introducing **tiling** and the **online Softmax** trick. Instead of computing the entire matrix at once, it breaks the Queries (Q), Keys (K), and Values (V) into smaller blocks (tiles) that fit into the GPU's ultra-fast SRAM.

Algorithm 1 FlashAttention (Forward Pass)

Require: Matrices $Q, K, V \in R^{N \times d}$ in HBM, SRAM size M.
Set block sizes $B_c = \frac{M}{4d}$, $B_r = \min(\frac{M}{4d}, d)$.
Initialize $O = (0)^{N \times d}$, $\ell = (0)^N$, $m = (-\infty)^N$ in HBM.
Divide Q, K, V, O, ℓ, m into blocks of size B_r and B_c.
for $1 \leq j \leq T_c$ **do**
  Load K_j, V_j from HBM to SRAM.
  for $1 \leq i \leq T_r$ **do**
    Load Q_i, O_i, ℓ_i, m_i from HBM to SRAM.
    On chip, compute $S_{ij} = Q_i K_j^T$.
    On chip, compute $\tilde{m}_{ij} = \mathrm{rowmax}(S_{ij})$, $\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij})$, $\tilde{\ell}_{ij} = \mathrm{rowsum}(\tilde{P}_{ij})$.
    On chip, update $m_i^{\mathrm{new}} = \max(m_i, \tilde{m}_{ij})$ and $\ell_i^{\mathrm{new}} = e^{m_i - m_i^{\mathrm{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\ell}_{ij}$.
    Write $O_i \leftarrow \mathrm{diag}(\ell_i^{\mathrm{new}})^{-1} \left(\mathrm{diag}(\ell_i) e^{m_i - m_i^{\mathrm{new}}} O_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{P}_{ij} V_j \right)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\mathrm{new}}$, $m_i \leftarrow m_i^{\mathrm{new}}$ to HBM.
  end for
end for
return O

As outlined in Algorithm 1, the outer loops stream blocks of K and V, while the inner loops stream blocks of Q, continuously updating the running maximum (m_i) and running sum (ℓ_i). This guarantees mathematically exact attention without ever writing the intermediate S or P matrices to HBM.

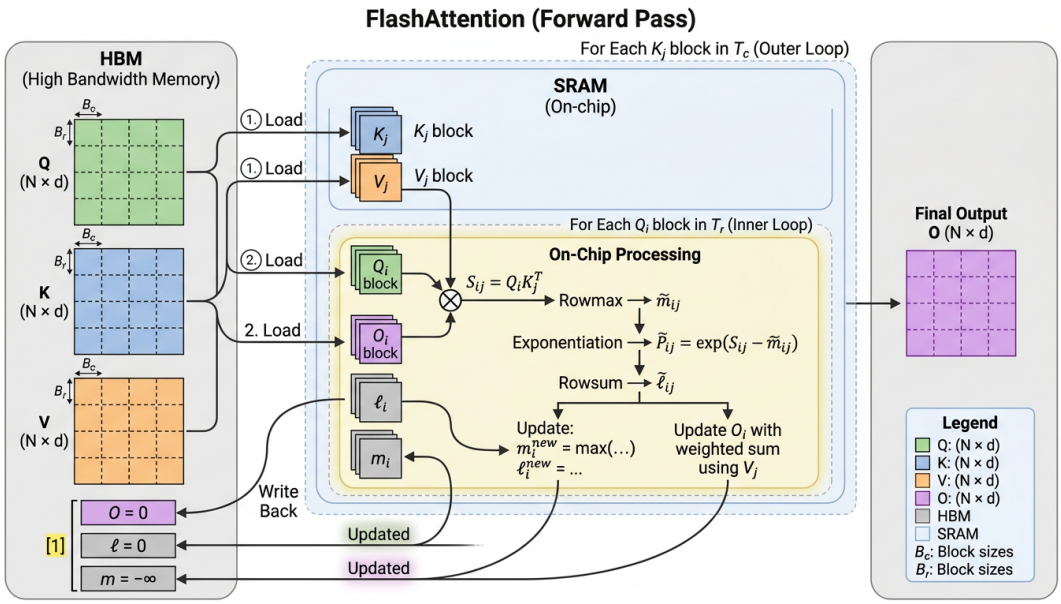

Figure 1: Conceptual diagram of FlashAttention. Tiling allows the attention matrix and Softmax to be computed entirely in SRAM, avoiding costly HBM accesses.

4 Methodology

Our approach involves replacing the standard PyTorch `DynamicCache` and `scaled_dot_product_attention` (SDPA) with a custom `QuantizedKVCache` and a fused Triton attention kernel.

4.1 Hardware-Aware Kernels via OpenAI Triton

Writing the complex loops and memory management of Algorithm 1 in CUDA C++ is notoriously difficult. In this project, we utilize OpenAI's Triton [4]. Triton provides a Python-based programming model that abstracts away thread-level execution blocks. Instead, programmers operate on block-level tensors, and the Triton compiler automatically optimizes the memory access patterns and parallelization.

4.2 Fused INT4 Dequantization Decode Algorithm

By using Triton, we were able to extend the FlashAttention algorithm specifically for the decode stage (where the Query sequence length is 1). We completely bypass the standard Hugging Face pipeline that requires dequantizing the KV cache back to FP16 in HBM. Instead, we fuse our custom INT4 dequantization directly into the inner SRAM loop of the decoding kernel.

Algorithm 2 Fused INT4 Dequantized Flash-Decoding (Decode Stage)

Require: Query vector $q \in R^{1\times d}$ in HBM.
Require: Packed INT4 Keys $K_{\text{packed}} \in U^{N\times(d/2)}$, Values $V_{\text{packed}} \in U^{N\times(d/2)}$ in HBM.
Require: FP16 Scales $K_{\text{scales}}, V_{\text{scales}}$ for grouped quantization in HBM.
Initialize $O = (0)^{1\times d}$, $\ell = 0$, $m = -\infty$ in SRAM.
Load q into SRAM.
for each block j of KV cache in HBM **do**
  Load block $K_{\text{packed},j}$ and $K_{\text{scales},j}$ into SRAM.
  On-chip Dequantize: Extract 4-bit nibbles from $K_{\text{packed},j}$, convert to float32, and multiply by $K_{\text{scales},j}$ to obtain $K_j \in R^{B\times d}$.
  On chip, compute $S_j = qK_j^T$.
  On chip, compute $\tilde{m}_j = \max(S_j)$, $\tilde{P}_j = \exp(S_j - \tilde{m}_j)$, $\tilde{\ell}_j = \sum(\tilde{P}_j)$.
  On chip, update running $m^{\text{new}} = \max(m, \tilde{m}_j)$ and $\ell^{\text{new}} = e^{m-m^{\text{new}}}\ell + e^{\tilde{m}_j - m^{\text{new}}}\tilde{\ell}_j$.
  Load block $V_{\text{packed},j}$ and $V_{\text{scales},j}$ into SRAM.
  On-chip Dequantize: Extract nibbles from $V_{\text{packed},j}$ and apply $V_{\text{scales},j}$ to obtain $V_j \in R^{B\times d}$.
  Update running output: $O \leftarrow e^{m-m^{\text{new}}}O + e^{\tilde{m}_j - m^{\text{new}}}\tilde{P}_j V_j$.
  $\ell \leftarrow \ell^{\text{new}}$, $m \leftarrow m^{\text{new}}$.
end for
return $\text{diag}(\ell)^{-1}O$ to HBM.

As shown in Algorithm 2, the cache is read as packed `uint8` tensors. The dequantization to float happens entirely inside the GPU's fast registers (SRAM) right before the dot product is calculated. The full-precision KV block is then immediately discarded, preventing any heavy writes to HBM.

4.3 System Integration

- **System Design**: We intercept the Hugging Face model execution by swapping the decoder-layer attention modules to `LlamaAttentionQuantized` in place. We preserve the Hugging Face `accelerate` hooks via weight transplantation to ensure seamless multi-GPU sharding (e.g., `device_map="auto"`).

- **Algorithms**: The custom `QuantizedKVCache` dynamically quantizes the new token's K and V and appends it to the packed cache during generation.

- **Hardware Compatibility**: The stack relies on PyTorch and Triton, supporting Ampere/Hopper architectures and providing fallback `sm70` kernels for older architectures like

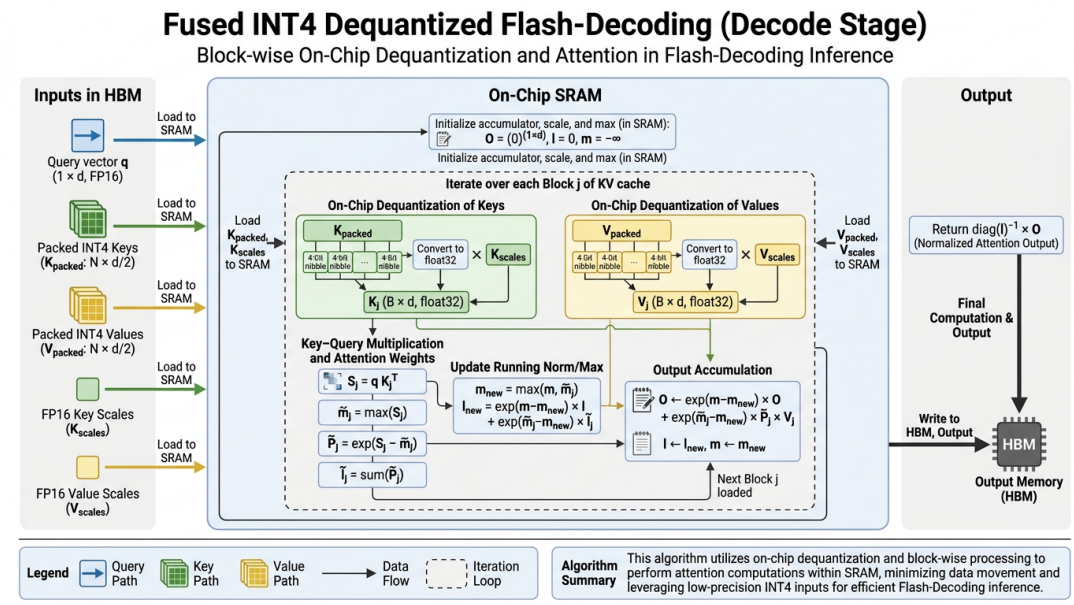

Figure 2: Conceptual diagram of fused INT4 dequantization during Flash-Decoding. Packed INT4 KV blocks are loaded from HBM, dequantized on chip, and immediately consumed by the attention computation in SRAM.

the V100.

5 Experimental Setup

- **Hardware**: NVIDIA RTX PRO 6000 Blackwell Server Edition GPUs with 96 GB VRAM each on Google Colab
- **Software Stack**: Python, PyTorch, CUDA, Triton, and Hugging Face Transformers.
- **Models & Workloads**: `meta-llama/Llama-3.1-8B`, `meta-llama/Llama-3.1-70B`, and GPT-2 models. Workloads include standard generation, WikiText perplexity, LongBench (e.g., NarrativeQA, TriviaQA), and Passkey retrieval.
- **Baselines**: The primary baseline is the stock PyTorch `DynamicCache` (BF16 KV) using native SDPA.
- **Metrics**: We measure Decode Latency (ms/token), Peak VRAM (MiB), KV Cache Storage (MiB), Throughput, Perplexity, and Logit Agreement.

6 Results and Analysis

All experiments were conducted on a single NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (96 GB VRAM, ∼1.8 TB/s HBM bandwidth). Both models were loaded in BF16 . The INT4 quantized path uses a group size of 32. Each decode benchmark consists of 64 autoregressive decode steps after prefill, with 1 warmup iteration.

KV cache storage. INT4 KV quantization substantially reduces KV tensor storage. For 8B, KV storage drops from 136 MiB to 38.25 MiB at 1024 tokens, from 520 MiB to 146.25 MiB at 4096 tokens, and from 2056 MiB to 578.25 MiB at 16384 tokens. For 70B, KV storage drops from 340 MiB to 95.625 MiB at 1024 tokens and from 1300 MiB to 365.625 MiB at 4096 tokens.

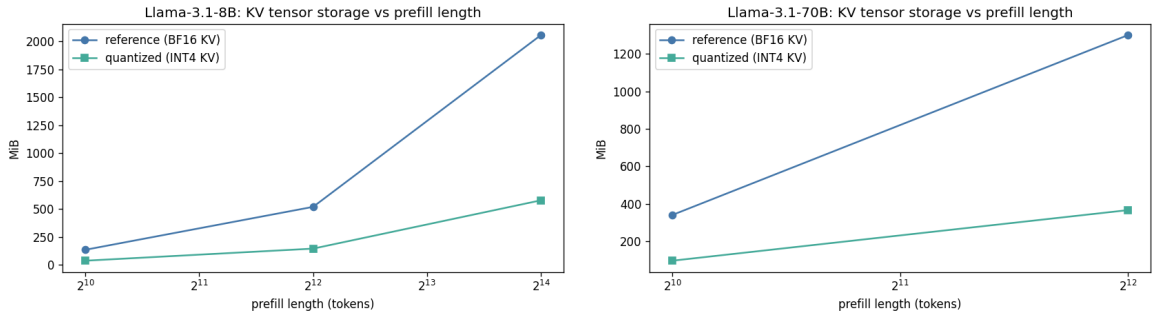

Figure 3: KV cache tensor storage (MiB) across prefill lengths for LLaMA 3.1 8B (left) and 70B (right).

Peak VRAM. Peak allocated VRAM is slightly lower with INT4 KV in this run. For 8B, peak VRAM decreases from 15,711 MiB to 15,619 MiB at 1024 tokens and from 21,509 MiB to 20,037 MiB at 16384 tokens. For 70B, peak VRAM decreases from 86,204 MiB to 85,974 MiB at 1024 tokens and from 87,964 MiB to 87,044 MiB at 4096 tokens. While peak VRAM includes more than the KV cache, these results are consistent with KV memory savings reducing overall memory pressure.

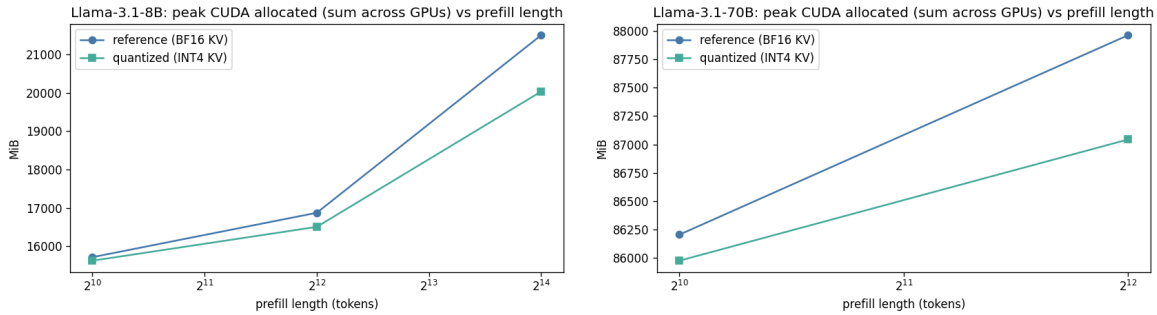

Figure 4: Peak CUDA allocated (MiB) across prefill lengths for LLaMA 3.1 8B (left) and 70B (right).

Decode latency. Decode latency (ms/token) varies by model size and prefill length. For 8B, the INT4 KV path is slower than the BF16 baseline in this configuration: 14.59 ms/token (ref) vs 17.95 ms/token (INT4) at 1024 tokens, 15.34 vs 28.33 at 4096 tokens, and 17.23 vs 68.25 at 16384 tokens. For 70B, decode latency is similar at 1024 tokens (2176.38 vs 2177.10 ms/token) and increases modestly at 4096 tokens (2119.44 vs 2195.95 ms/token). These results suggest that while INT4 KV reduces memory footprint, realized speedups depend on whether decode is dominated by KV bandwidth versus additional overheads (e.g., dequantization, kernel launch behavior, and GPU architecture effects).

Quality (perplexity). Table 1 reports WikiText perplexity. For both 8B and 70B, INT4 KV increases perplexity slightly relative to the reference cache, indicating a small accuracy cost in exchange for the memory savings.

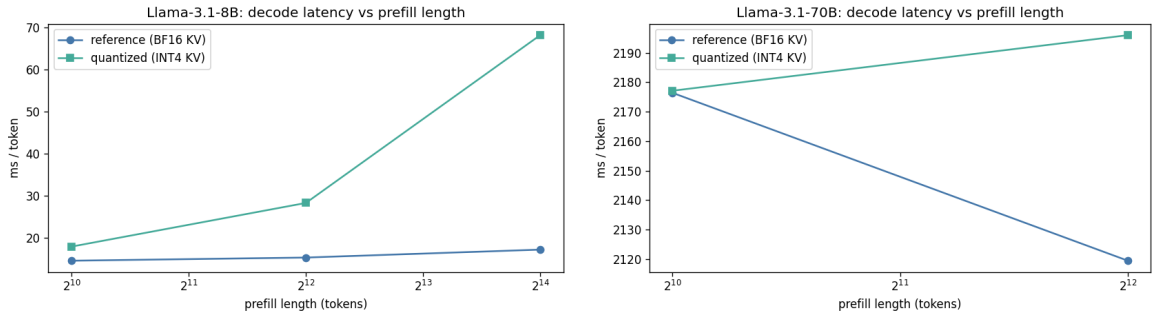

Figure 5: Decode latency (ms/token) across prefill lengths for LLaMA 3.1 8B (left) and 70B (right).

Model	Reference PPL	INT4 KV PPL
LLaMA 3.1 8B	11.87	12.20
LLaMA 3.1 70B	7.36	7.57

Table 1: WikiText perplexity (lower is better).

7 Bottleneck/Profiling Analysis

7.1 Why Decode Latency is Higher

There are 2 primary reasons for the elevated decode latency in the quantized path:

1. **GPU Underutilization**: Our kernel launches `grid = (B, n_q_heads, 1)` = 32 thread blocks for the 8B model. A modern GPU with 76+ SMs can only utilize ∼42% of its compute capacity. FlashAttention employs *Split-K* (Flash-Decoding) to parallelize across the KV sequence dimension, achieving much higher SM occupancy.

2. **Dequantization Overhead**: Each inner-loop iteration performs additional memory loads (quantization scales) and ALU operations (nibble extraction, bias subtraction, scale multiplication) that FlashAttention does not require.

8 Reproducibility

The project code is located at `github.com/johnchmura/kv-cache-triton`. To reproduce the LLaMA 3.1 results:

```
pip install -r requirements.txt
export HF_HOME=/path/to/hf_cache
CUDA_VISIBLE_DEVICES=0 python -m benchmarks.llama3.run_llama3_benchmark \
    --models meta-llama/Llama-3.1-8B \
    --prefill-lens 1024 4096 16384 \
    --ppl-samples 32 --continue-on-error
```

The outputs are automatically saved in timestamped directories under `benchmarks/llama3/logs/`, containing JSON metrics, CSV summaries, and PNG plots for latency, memory, and quality.

9 Limitations and Future Work

- **Decode Latency**: The fused INT4 kernel is slower than FlashAttention for batch-size-1 inference at moderate sequence lengths (<32K tokens). The primary bottleneck is

GPU underutilization (32 thread blocks on 76 SMs). Implementing *Split-K parallelization* (Flash-Decoding) would distribute KV cache processing across all SMs and is expected to close most of the latency gap.

- **Batch Size**: Our benchmarks only evaluate batch-size-1. Prior work shows that KV cache quantization provides significant latency benefits at batch sizes ≥ 8, where cache bandwidth dominates the decode step.
- **TF32 Incompatibility**: Disabling TF32 reduces tensor core throughput. Future work could explore quantization-aware rounding schemes that produce TF32-safe dequantized values.
- **Prefill Kernel**: The current Triton kernel is decode-only. A fused INT4 prefill kernel could reduce prefill latency for scenarios where the KV cache is pre-populated (e.g., prompt caching).

10 Team Contribution Statement

- **John Chmura**: System architecture, Triton kernel implementation, INT4 quantization scheme, benchmark framework, multi-GPU integration.
- **Akash Madhu**: Kernel debugging and numerical analysis (TF32 precision exploration, BLOCK_K tuning, BF16 tensor core optimization), , profiling and bottleneck analysis.

References

[1] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Zhang, H. (2023). *Efficient memory management for large language model serving with pagedattention.* SOSP 2023.

[2] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.* arXiv preprint arXiv:2306.00978.

[3] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* arXiv preprint arXiv:2210.17323.

[4] Tillet, P., Kung, H. T., & Cox, D. (2019). *Triton: an intermediate language and compiler for neural net accelerators.* MAPL 2019.

[5] Dao, T., Fu, D., Ermon, S., Rudra, A., & Ré, C. (2022). *Flashattention: Fast and memory-efficient exact attention with io-awareness.* Advances in Neural Information Processing Systems, 35, 16344-16359.